



UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DEL NORTE (CUNOR)
FACULTAD DE INGENIERÍA ICS



SECUREFINANCE ERP

Documentación técnica y funcional

INTEGRANTES

Kenneth Christian Arnaldo Téllez Castillo

Jose Miguel de León Aguilar

Edward Rene Winter Tiu

Rubén Adolfo Herrera Morales

Curso: Base de Datos 2

Cobán, Alta Verapaz

2026



Contenido

Introducción.....	3
Objetivos.....	3
Alcance.....	3
Descripción general.....	4
Tecnologías.....	4
Arquitectura.....	5
Modelo de datos	5
Seguridad e identidad.....	5
Negocio y auditoría.....	6
Diagrama entidad-relación.....	6
Seguridad.....	8
Registro y autenticación	8
Datos sensibles.....	8
Control de acceso RBAC.....	9
Facturación.....	10
Ejemplo con precios	10
Transacciones ACID y concurrencia.....	11
Atomicidad y rollback.....	11
Propiedades ACID	11
Concurrencia.....	11
Auditoría.....	11
Funciones SQL.....	12
Histórico de ventas	12
Interfaz	12
Instalación	13
Requisitos y preparación.....	13
Instalación Full.....	13
Instalación modular.....	13
Permisos del principal de aplicación.....	14
Pruebas.....	14
Matriz de pruebas.....	14
Manual de usuario.....	14
Manual técnico.....	15
Rutas web.....	15
Limitaciones	16
Conclusiones	16
Recomendaciones	16
Contribuciones del equipo.....	17



Introducción

Este documento describe el sistema que existe en el repositorio SecureFinance-ERP-BD2: sus capas, objetos SQL, reglas de facturación, controles de acceso, instalación, pruebas y operación. Su alcance es académico y documenta el comportamiento comprobado en archivos y scripts; distingue entre pruebas automatizadas ejecutadas y evidencia que aún requiere SQL Server o capturas reales.

El diseño concentra la validación de datos, el cálculo monetario y las transacciones en SQL Server, mientras Express entrega las páginas y coordina llamadas con parámetros tipados. La aplicación evita acceso directo de lectura o escritura a tablas para su cuenta operativa.

Objetivos

- Documentar la estructura funcional y técnica de SecureFinance ERP a partir de main.
- Explicar el esquema relacional, RBAC, la facturación atómica y la trazabilidad.
- Presentar instrucciones reproducibles de instalación y operación.
- Registrar el estado real de las pruebas y de las evidencias, sin declarar validaciones no ejecutadas.

Alcance

El sistema cubre inicio y cierre de sesión; consulta del dashboard; facturación de entre 1 y 100 productos; validación de cliente, usuario, productos, cantidades, duplicados y stock; generación de factura, detalle y movimiento de caja; consulta de bitácora de acceso, auditoría DML e histórico de ventas

El inventario final contiene 13 tablas, 10 procedimientos almacenados, 4 funciones y 3 triggers. También existe un tipo de tabla SQL, dbo.TipoDetalleVenta, que transporta las líneas de una venta multiproducto.



Descripción general

El usuario ingresa por el portal web. SQL Server valida la contraseña y registra cada intento. Tras autenticarse, la aplicación consulta los permisos efectivos y renueva la sesión. Los módulos de ventas y auditoría verifican sesión y permiso antes de cargar sus páginas. En una venta, SQL vuelve a validar los datos contra el estado actual, bloquea las filas de producto, calcula los importes y confirma todos los cambios dentro de una transacción.

La identidad visual se presenta como un escritorio de operaciones financieras, con navegación lateral, indicadores y módulos de Facturación y Auditoría visibles según el permiso de sesión.

Tecnologías

Área	Tecnología confirmada	Uso en el proyecto
Servidor	Node.js 24 o posterior; Express 5	Rutas HTTP, middleware, sesión y manejo de errores.
Vistas	EJS	Renderizado de login, dashboard, facturación y auditoría.
Interfaz	Bootstrap 5.3, CSS propio, JavaScript del navegador	Diseño adaptable, estilos corporativos y preparación del detalle de venta.
SQL	Microsoft SQL Server 2016 SP1 o posterior	Modelo relacional, SP, funciones, triggers, transacciones y permisos.
Driver	mssql 12	Pool compartido y parámetros SQL tipados.
Pruebas	node:test; suites T-SQL; sqlcmd opcional	Pruebas Node con mocks y validaciones de base de datos en ambiente aislado.
Seguridad	SHA2_512 + salt individual, RBAC, sesiones, CSRF, mínimo privilegio	Autenticación, autorización y protección de operaciones web/SQL.

El nombre de instancia de SQL Server no se concatena al host: se configura DB_SERVER y el puerto TCP real en DB_PORT. El driver utiliza cifrado por defecto; confiar en certificado autofirmado se reserva para el entorno local de desarrollo.



Arquitectura

SecureFinance ERP | Arquitectura por capas

La aplicación obtiene importes definitivos de SQL-Server y se conecta con securefinance_app

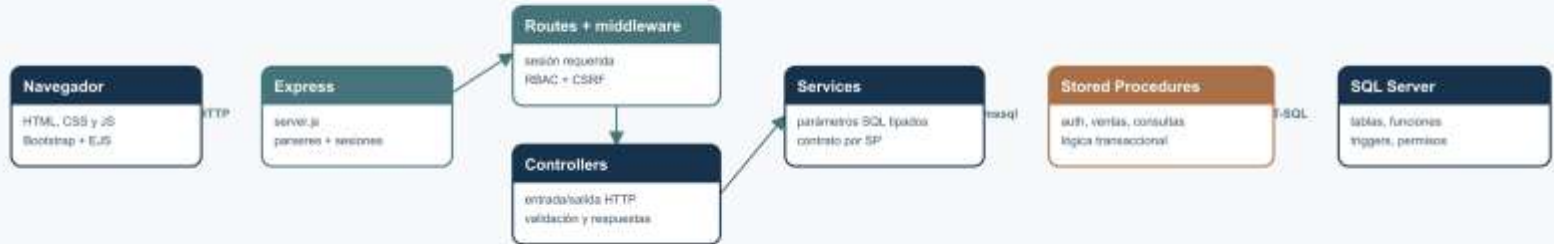


Figura 1. Capas web, acceso a datos y SQL Server

Express configura EJS, los recursos estáticos, los límites del cuerpo HTTP y express-session. Las rutas entregan la solicitud al controlador tras aplicar sesión y RBAC; el servicio convierte los valores admitidos en parámetros tipados y ejecuta procedimientos. El pool mssql se comparte entre solicitudes y el servidor valida su conexión antes de escuchar en 127.0.0.1:3000.

La sesión se conserva en MemoryStore, con cookie httpOnly, sameSite lax y duración de 30 minutos. Al autenticar, se regenera el identificador para evitar fijación de sesión. La facturación añade un token CSRF aleatorio de 32 bytes ligado a la sesión y lo compara con timingSafeEqual antes de invocar SQL.

Modelo de datos

El esquema separa identidad y autorización, operación comercial y auditoría. Las claves foráneas impiden registrar ventas para clientes o usuarios inexistentes, detalles para facturas/productos inexistentes y movimientos de caja para facturas inexistentes. Las tablas puente tienen clave primaria compuesta para impedir asignaciones repetidas.

Seguridad e identidad

Tabla	Propósito y clave	Relaciones y controles relevantes
Usuario	Identidad y credenciales; PK UsuarioId.	NombreUsuario, Correo y Salt únicos; hash de 64 bytes y salt de 32; Activo y DebeCambiarPassword.
Rol	Catálogo de roles; PK RolId.	Nombre único; Activo.
Permiso	Catálogo de permisos; PK PermisoId.	Codigo único; Activo.
Usuario_Rol	Asignación Usuario↔Rol; PK (UsuarioId, RolId).	FK a Usuario y Rol; FechaAsignacion UTC.
Rol_Permiso	Asignación Rol↔Permiso; PK (RolId, PermisoId).	FK a Rol y Permiso.
Token_Recuperacion	Esquema para tokens de recuperación; PK TokenRecuperacionId.	FK UsuarioId; guarda TokenHash, fechas y estado; no existe el flujo web completo.
Bitacora_Acceso	Registro de resultados de login; PK BitacoraAccesoId.	FK UsuarioId nullable; resultado limitado a cuatro estados; fecha UTC y metadatos SQL.



Negocio y auditoría

Tabla	Propósito y clave	Relaciones y controles relevantes
Cliente	Catálogo de clientes; PK ClienteId.	NIT único; Factura lo referencia. Nombre y NIT no vacíos.
Producto	Catálogo con precio y stock; PK ProductoId.	Código único; Precio ≥ 0 ; Stock ≥ 0 ; Factura lo referencia por DetalleFactura.
Factura	Cabecera de venta; PK FacturaId.	FK ClienteId y UsuarioId; importes no negativos; Total = Subtotal + IVA; estado EMITIDA.
DetalleFactura	Líneas con cantidad, precio y subtotal; PK DetalleFacturaId.	FK FacturaId y ProductoId; UNIQUE (FacturaId, ProductoId); cantidad > 0 .
MovimientoCaja	Ingreso asociado a la venta; PK MovimientoCajaId.	FacturaId es FK y UNIQUE, por ello una factura tiene como máximo un movimiento; tipo INGRESO.
Bitacora_Transacciones	Eventos de triggers; PK BitacoraTransaccionId.	Sin FK; operación INSERT/UPDATE/DELETE, JSON anterior/nuevo e identidad SQL/contexto.

El TVP dbo.TipoDetalleVenta (ProductoId INT, Cantidad INT) no es una tabla persistente del modelo: es un tipo estructurado de entrada para sp_ProcesarVentaTransaccional.

Diagrama entidad-relación

SecureFinance ERP | Modelo relacional general

13 tablas | líneas = FK | 1:N salvo relación Factura-MovimientoCaja (máximo uno por factura)



Figura 2. ERD general con las 13 tablas y relaciones



SecureFinance ERP | Seguridad y RBAC

Las dos tablas puente resuelven las relaciones N:M

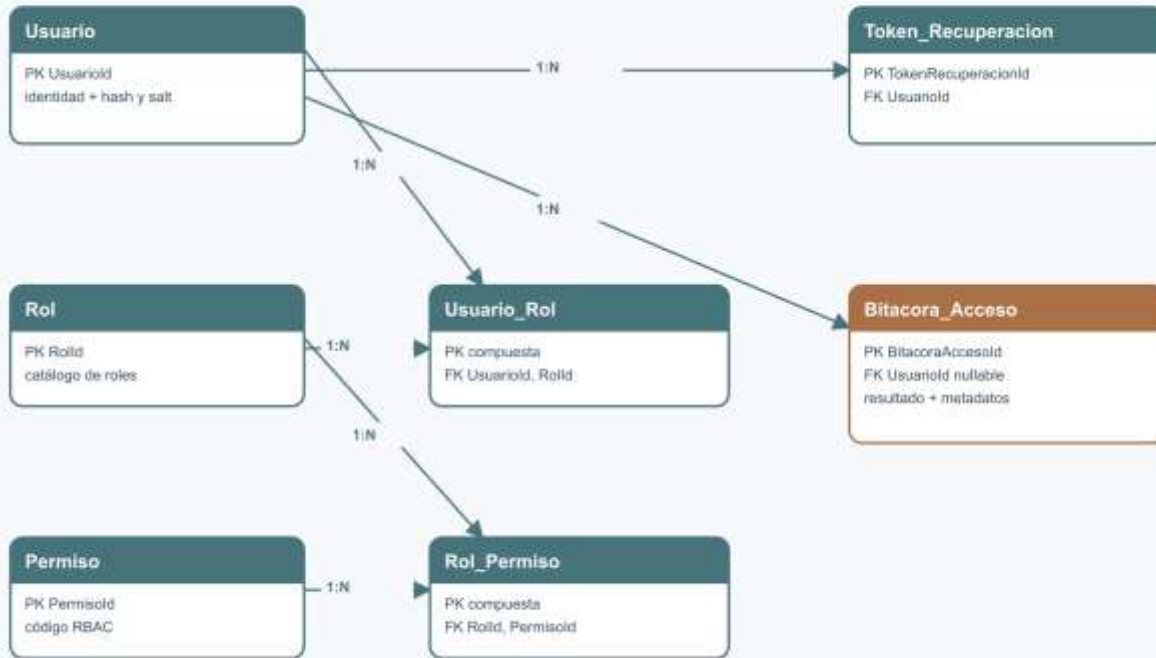


Figura 3. ERD de identidad, roles y permisos

SecureFinance ERP | Negocio y facturación

Factura tiene líneas de detalle; cada producto puede aparecer en varias líneas de facturas

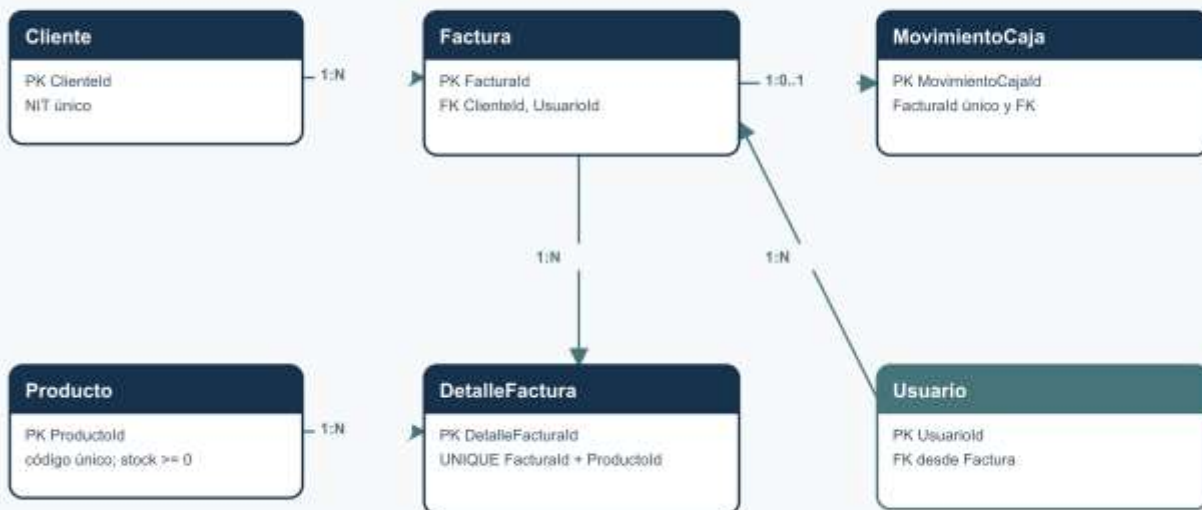


Figura 4. ERD de negocio y facturación



La cardinalidad Usuario↔Rol y Rol↔Permiso es N:M y queda materializada por Usuario_Rol y Rol_Permiso. Un cliente puede aparecer en varias facturas; una factura tiene varias líneas; un producto puede aparecer en varias líneas. MovimientoCaja tiene una relación 1:0..1 con Factura porque FacturaId es único en esa tabla. Bitacora_Transacciones conserva identificadores y valores del evento, sin FK a las filas auditadas.

Seguridad

Registro y autenticación

dbo.sp_RegistrarUsuario valida longitudes, presencia y unicidad; genera 32 bytes con CRYPT_GEN_RANDOM(32) y calcula HASHBYTES('SHA2_512', CONVERT(VARBINARY(256), @Password) + @Salt). SQL Server almacena PasswordHash (64 bytes) y Salt (32 bytes). Node no calcula ni devuelve hash o salt.

La contraseña se conserva como NVARCHAR y SQL usa su representación binaria UTF-16LE junto con el salt; el procedimiento no recorta espacios ni normaliza el valor.

dbo.sp_Login busca un usuario activo, calcula el mismo hash y compara el resultado. Registra el intento con dbo.sp_RegistrarAcceso: EXITOSO, PASSWORD_INCORRECTA, USUARIO_INEXISTENTE o USUARIO_INACTIVO.

Si falla la escritura de auditoría, el procedimiento no declara el acceso exitoso. La bitácora registra la conexión del backend a SQL Server, por lo que IpConexionSQL no identifica necesariamente el navegador.

Las vistas protegidas no se sirven desde caché. La ruta de logout destruye la sesión y limpia la cookie. Los errores del driver, los parámetros y las credenciales no se devuelven al usuario web.

Datos sensibles

Bitacora_Transacciones registra columnas seleccionadas de Producto y Factura en JSON. Los scripts no incluyen PasswordHash, Salt, TokenHash, contraseñas ni secretos dentro de ValorAnterior o ValorNuevo. La bitácora de acceso no almacena la contraseña enviada.



Control de acceso RBAC

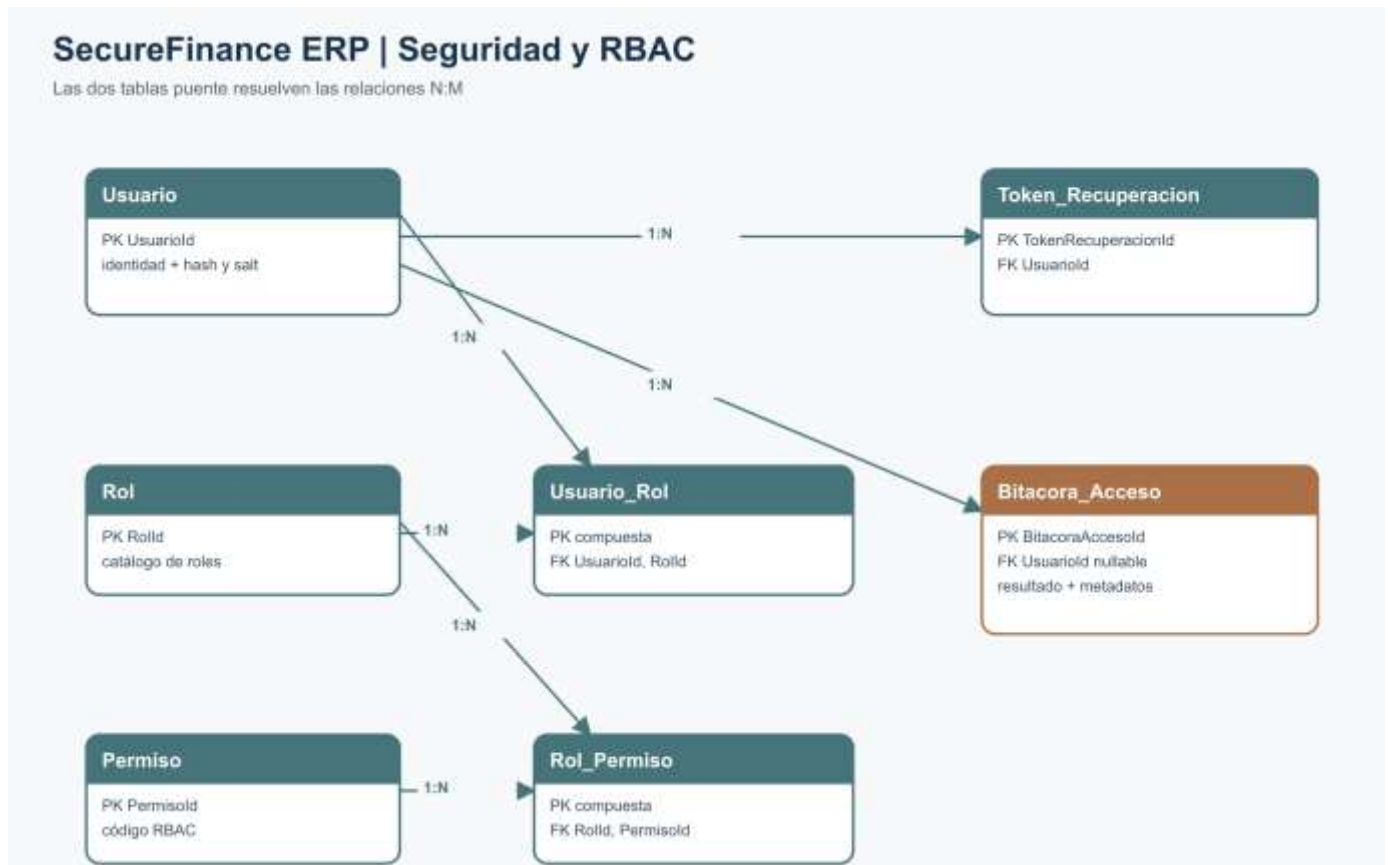


Figura 5. Asignación Usuario > Usuario_Rol > Rol > Rol_Permiso > Permiso

Rol	Permisos asignados por semilla	Uso de rutas observado
Administrador	USUARIOS_ADMINISTRAR; VENTAS_REGISTRAR; AUDITORIA_CONSULTAR	Facturación y auditoría disponibles; no hay módulo web de administración de usuarios.
Cajero	VENTAS_REGISTRAR	Puede acceder a facturación; auditoría responde 403.
Auditor	AUDITORIA_CONSULTAR	Puede consultar auditoría; facturación responde 403.

sp_ObtenerPermisosUsuario devuelve códigos efectivos distintos únicamente de roles y permisos activos y de un usuario activo. Node guarda esos códigos en la sesión tras el login; para reflejar cambios RBAC el usuario debe iniciar sesión de nuevo. Aunque USUARIOS_ADMINISTRAR existe en la semilla, la versión no incluye una ruta, controlador o vista de administración de usuarios.



Facturación

El formulario obtiene clientes y productos activos mediante procedimientos almacenados. JavaScript mantiene el detalle de hasta 100 productos, acumula una selección repetida como una sola línea, muestra cantidad, precio, subtotal e IVA preliminar y deshabilita el envío duplicado mientras procesa. El servidor acepta únicamente ClienteId y pares ProductoId/Cantidad; ignora UsuarioId, precios e importes enviados por el navegador. El UsuarioId procede de la sesión.

El usuario autenticado selecciona un cliente activo.

Agrega uno o más productos y cantidades enteras positivas; la página valida los datos preliminares y no admite más de 100 productos diferentes.

El servidor verifica token CSRF, tipos, límites y productos duplicados antes de llamar al servicio.

sp_ProcesarVentaTransaccional vuelve a validar cliente, usuario, productos y stock actual bajo una transacción.

SQL bloquea el stock y el precio; después actualiza existencias, calcula importes, crea Factura, DetalleFactura y un MovimientoCaja tipo INGRESO.

Si todas las operaciones terminan, SQL confirma con COMMIT y devuelve identificador y totales calculados.

SQL Server es la autoridad de precios y de los importes definitivos. El resumen del navegador es preliminar y usa centavos enteros para evitar errores de coma flotante; tras procesar, la confirmación muestra los valores devueltos por SQL.

Ejemplo con precios

Producto	Cantidad × precio	Subtotal de línea
Teclado USB	2 × Q 125.50	Q 251.00
Mouse óptico	3 × Q 75.25	Q 225.75
Subtotal		Q 476.75
IVA 12% sobre subtotal global	ROUND(476.75 × 0.12, 2)	Q 57.21
Total	Subtotal + IVA	Q 533.96

Estos importes corresponden a los precios originales de las semillas DEMO. SQL Server calcula los valores definitivos dentro de sp_ProcesarVentaTransaccional y redondea el IVA del subtotal global a dos decimales.



Transacciones ACID y concurrencia

Atomicidad y rollback

El procedimiento activa SET XACT_ABORT ON, inicia BEGIN TRANSACTION y encapsula el flujo en TRY/CATCH. Ante error ejecuta ROLLBACK cuando XACT_STATE() no es cero, limpia el parámetro de salida FacturaId y vuelve a lanzar el error con THROW. SQL Server no conserva una venta parcial: el descuento de stock, la factura, los detalles, el ingreso de caja y los registros de trigger quedan sujetos a la misma transacción.

Si un producto no existe, está inactivo o no tiene existencias suficientes, el error cancela toda la venta. El contrato del procedimiento define 52010 para stock insuficiente. No se afirma que el rollback se haya observado en una instancia SQL durante esta revisión; la suite TransactionTests.sql contiene esa comprobación y queda pendiente de ejecución en un SQL Server accesible.

Propiedades ACID

Propiedad	Aplicación real en el sistema
Atomicidad	La venta completa se confirma o revierte; no se conserva factura, caja o stock parcial.
Consistencia	PK/FK, UNIQUE, CHECK y validaciones del procedimiento mantienen reglas de cantidad, precio, total y estado.
Aislamiento	UPDLOCK/HOLDLOCK serializa la lectura del stock y precio por producto mientras la venta está abierta; el acceso se hace en orden ascendente de ProductoId.
Durabilidad	COMMIT persiste los cambios según las garantías transaccionales de SQL Server; las pruebas actuales no constituyen una prueba de recuperación ante fallos de servidor.

Concurrencia

El TVP puede llegar en cualquier orden, pero el procedimiento busca repetidamente el ProductoId siguiente y bloquea la fila en orden ascendente. El acceso puntual por clave primaria con UPDLOCK y HOLDLOCK conserva el stock y el precio leídos hasta COMMIT y reduce carreras entre ventas que compiten por el mismo producto. La aplicación no reintenta automáticamente una solicitud con resultado incierto: un timeout podría ocurrir después del COMMIT y un reenvío duplicaría la venta.

Auditoría

Objeto	Responsabilidad
Bitacora_Acceso	Intentos de autenticación con usuario intentado, resultado, hora UTC, HostName, AppName, UsuarioSQL e IP de la conexión SQL.
Bitacora_Transacciones	Cambios seleccionados de negocio con tabla, operación, contexto SQL, identificador y valores JSON anteriores/nuevos.
tr_Producto_Auditar_Update	Audita solo cambios en Precio o Stock; serializa campos seleccionados en ValorAnterior y ValorNuevo.
tr_Producto_Auditar_Delete	Registra el estado eliminado de Producto; ValorNuevo queda NULL.
tr_Factura_Auditar_Insert	Registra la factura insertada; ValorAnterior NULL y ValorNuevo con sus datos de cabecera.
sp_ConsultarBitacoraAcceso	Filtra por fecha inicial/final, usuario intentado y resultado.
sp_ConsultarAuditoriaTransacciones	Filtra por fecha inicial/final, tabla y operación usando fn_ConsultarAuditoria.



Bitacora_Transacciones conserva FechaHora, TablaAfectada, Operacion, UsuarioSQL, HostName, AppName, IdentificadorRegistro, ValorAnterior y ValorNuevo. Los eventos INSERT/DELETE usan NULL en el lado que no existe; Producto UPDATE guarda ambos estados como JSON.

La autenticación se registra exclusivamente en Bitacora_Acceso; se elimina un trigger previo de Usuario para evitar duplicar esa función. UsuarioSQL identifica la cuenta que conectó a SQL Server (en la aplicación, la cuenta dedicada), no necesariamente la persona autenticada en la web. La aplicación y las vistas no deben mostrar secretos ni guardar PasswordHash, Salt, TokenHash, contraseñas o tokens sin hash en estas bitácoras.

Funciones SQL

Función	Tipo y responsabilidad
fn_CalcularIVA	Escalar; retorna el 12% de un monto con tipo DECIMAL(18,2).
fn_CalcularSubtotal	Escalar; multiplica cantidad por precio unitario y devuelve DECIMAL(18,2).
fn_ConsultarAuditoria	Table-valued; consulta bitácora DML por rango de fecha, tabla y operación.
fn_ObtenerHistoricoVentas	Table-valued; reúne factura, fecha, cliente, usuario, subtotal, IVA y total con filtros.

El procedimiento transaccional actual calcula el subtotal y el IVA directamente y no invoca fn_CalcularIVA ni fn_CalcularSubtotal; el código del SQL señala estas funciones como punto futuro de integración.

Histórico de ventas

fn_ObtenerHistoricoVentas une Factura con Cliente y Usuario para presentar Factura, Fecha, Cliente, Usuario, Subtotal, IVA y Total. Acepta FechaInicial, FechaFinal y texto Cliente; la búsqueda usa CHARINDEX en Nombre. sp_ConsultarHistoricoVentas devuelve esos campos ordenados por Fecha descendente y luego Factura descendente.

La página /auditoria muestra el histórico junto con la bitácora de acceso y la auditoría DML. La fecha inicial/final se aplica a las tres consultas; Cliente filtra solo el histórico. En auditoría DML también se aplican Tabla y Operacion. En accesos se aplican usuario intentado y resultado; los datos no se comparten como filtros entre consultas salvo fecha.

Interfaz

Vista	Elementos observados
Login	Usuario y contraseña; mensajes de error; no incluye flujo para recuperar contraseña.
Dashboard	Bienvenida, estado de sesión, conteo de permisos y tarjetas de módulos permitidos.
Facturación	Cliente, selección de producto, stock visible, cantidad, detalle editable, resumen preliminar en quetzales y confirmación SQL.
Auditoría	Filtros y tres conjuntos de resultados: accesos, eventos DML e histórico de ventas.

Las vistas comparten navegación lateral, CSS corporativo, etiquetas jerárquicas y componentes Bootstrap. La hoja de estilos define adaptación para ventanas estrechas; la tabla de detalle y los resultados extensos usan contenedores desplazables. Los textos de base de datos se imprimen con escape EJS.



Instalación

Requisitos y preparación

- Node.js 24 o posterior y npm.
- Microsoft SQL Server 2016 SP1 o posterior, con TCP/IP habilitado y puerto conocido.
- SQL Server Management Studio (SSMS) o sqlcmd.
- Login SQL dedicado securefinance_app creado por el administrador. Ningún instalador crea el login ni la contraseña.
- Clonar SecureFinance-ERP-BD2 y situarse en main.
- Instalar dependencias con npm ci.
- Crear el login SQL de servidor securefinance_app mediante administración local autorizada.
- Instalar el esquema con la opción Full o con todos los módulos, nunca ambas opciones en la misma base.
- Si el login se creó después de instalar Full, ejecutar 10_AppPermissions.sql.
- Crear .env a partir de .env.example; definir DB_SERVER, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, SESSION_SECRET, DB_ENCRYPT y DB_TRUST_SERVER_CERTIFICATE.
- Ejecutar npm run check y npm test; comprobar el entorno local con npm run check:db.
- Iniciar con npm start y abrir <http://localhost:3000/login>.

El README fija el bind local en 127.0.0.1:3000 (URL del navegador <http://localhost:3000/login>). Mantener DB_ENCRYPT=true. DB_TRUST_SERVER_CERTIFICATE=true solo se contempla para desarrollo local con certificado autofirmado. La cuenta de ejecución debe ser exclusiva de la aplicación; nunca debe utilizar sa.

Instalación Full

database/SecureFinanceERP_Full.sql contiene los diez módulos de instalación indicados a continuación, no incluye las suites de prueba y debe ejecutarse solo en una base nueva. No es un script de migración. Ejecutarlo con una cuenta administradora autorizada, detenerse ante errores SQL y no continuar lotes posteriores a un error. Si no existe securefinance_app, el Full muestra una advertencia; luego de crear manualmente el login, ejecutar 10_AppPermissions.sql.

Instalación modular

Orden	Archivo	Responsabilidad
1	01_CreateDatabase.sql	Crea la base SecureFinanceERP.
2	02_SecurityTables.sql	Crea las siete tablas de identidad, roles, permisos, tokens y acceso.
3	03_SecurityStoredProcedures.sql	Alta de usuario, login, registro de acceso y consulta RBAC.
4	04_SecuritySeedData.sql	Semillas académicas de roles, permisos y admin_demo.
5	05_BusinessTables.sql	Crea Cliente, Producto, Factura, DetalleFactura, MovimientoCaja y dbo.TipoDetalleVenta.
6	05_BusinessSeedData.sql	Agrega clientes y productos ficticios; no repone stock ni modifica registros existentes.
7	06_TransactionProcedures.sql	Catálogos y venta transaccional con TVP.
8	07_AuditCore.sql	Bitacora_Transacciones, funciones y procedimientos de auditoría/histórico.
9	08_AuditTriggers.sql	Triggers de Producto UPDATE/DELETE y Factura INSERT.
10	10_AppPermissions.sql	Concede permisos mínimos al principal securefinance_app.

07_TransactionPermissions.sql y 09_AuditPermissions.sql se conservan como scripts parciales/históricos y no forman parte de la instalación final recomendada. El núcleo de auditoría requiere las tablas de negocio; los triggers se crean después de Bitacora_Transacciones.



Permisos del principal de aplicación

10_AppPermissions.sql mapea el login securefinance_app a un usuario de base si falta y concede EXECUTE únicamente a estos ocho procedimientos: sp_Login, sp_ObtenerPermisosUsuario, sp_ListarClientes, sp_ListarProductosDisponibles, sp_ProcesarVentaTransaccional, sp_ConsultarBitacoraAcceso, sp_ConsultarAuditoriaTransacciones y sp_ConsultarHistoricoVentas. Sobre dbo.TipoDetalleVenta concede EXECUTE y REFERENCES para el TVP.

No concede sysadmin, db_owner, db_datareader, db_datawriter, CONTROL ni SELECT/INSERT/UPDATE/DELETE sobre tablas. Tampoco revoca permisos previos: el administrador debe comprobar que securefinance_app no herede derechos amplios ni membresías adicionales.

Pruebas

Verificación realizada en la revisión: la comprobación equivalente a npm run check terminó sin errores en sus 18 archivos JavaScript. Las 18 pruebas de npm test pasaron (0 fallos), ejecutadas con la suite Node y los servicios SQL simulados. Este resultado valida lógica y contratos de la aplicación bajo mocks, no la persistencia en SQL Server.

Se revisaron las suites database/tests/SecurityTests.sql, AuthenticationTests.sql, TransactionTests.sql y AuditTests.sql. No se ejecutaron: sqlcmd está instalado, pero no hubo una instancia SQL Server accesible en localhost. Tampoco se confirmó la conexión mssql usando la cuenta securefinance_app. Por eso sus resultados y las pruebas de integración SQL se marcan PENDIENTE.

Comando / suite	Alcance	Resultado observado
npm run check	Sintaxis de aplicación, pruebas y prueba de integración Node.	VALIDADO: 18/18 archivos sin errores de sintaxis; ejecutado mediante el script check del proyecto.
npm test	Autenticación, sesión, ventas, frontend, RBAC y auditoría con mocks.	VALIDADO: 18 pruebas, 18 pasaron, 0 fallaron.
SecurityTests.sql	Restricciones y modelo de seguridad SQL.	PENDIENTE: requiere SQL Server accesible.
AuthenticationTests.sql	Hash/salt, login, semillas y RBAC en SQL Server.	PENDIENTE: requiere SQL Server accesible.
TransactionTests.sql	Transacción, cálculos, stock y rollback.	PENDIENTE: requiere SQL Server accesible.
AuditTests.sql	Funciones, triggers, bitácoras e histórico.	PENDIENTE: requiere SQL Server accesible.
npm run check:db	Conexión mssql con configuración local.	PENDIENTE: falta .env y servidor SQL accesible.

Matriz de pruebas

La matriz editable completa se entrega en Matriz_Pruebas_SecureFinance_ERP.xlsx. El estado VALIDADO se reserva para casos comprobados en ejecución Node; los casos que requieren comprobar persistencia, permisos SQL, triggers o rollback real permanecen PENDIENTE. Algunos errores de venta se validaron únicamente por el mapeo HTTP de un error simulado.

Manual de usuario

La guía separada Manual_Usuario_SecureFinance_ERP.docx explica el inicio del sistema, autenticación, dashboard, facturación, cliente, productos, cantidades, revisión de importes, procesamiento, consulta de auditoría y cierre de sesión en lenguaje no técnico.



Manual técnico

Ruta	Responsabilidad
src/config	Configuración y pool mssql; lectura estricta de .env y parámetros de conexión.
src/routes	Asocia verbos y rutas HTTP con controladores y middleware de sesión/RBAC.
src/middleware	requireAuth redirige si no hay usuario en sesión; guestOnly limita páginas de login.
src/controllers	Valida solicitudes web, coordina servicios, arma vistas y respuestas; maneja errores públicos seguros.
src/services	Llama procedimientos almacenados usando tipos y parámetros mssql.
src/views	Plantillas EJS: login, dashboard, facturación y auditoría.
src/public	CSS personalizado, Bootstrap servido desde node_modules y JavaScript de preparación/envío de venta.
database	Instalador Full y scripts modulares de tablas, procedimientos, semillas, funciones, triggers y permisos.
database/tests	Suites SQL y prueba de integración transaccional en base temporal aislada.
tests	Pruebas Node de servicios/controladores, interfaz HTTP y lógica del navegador.

Rutas web

Método y ruta	Acceso y comportamiento
GET /	Redirige a /dashboard si hay sesión; en otro caso, a /login.
GET /login	Solo invitado; muestra el formulario.
POST /login	Solo invitado; valida usuario/contraseña y registra acceso por SQL.
GET /dashboard	Requiere sesión; muestra permisos y módulos habilitados.
GET /logout	Destruye la sesión y limpia la cookie.
GET /facturacion	Requiere sesión y VENTAS_REGISTRAR; carga catálogos activos.
POST /facturacion/procesar	Requiere sesión, permiso, token CSRF y detalle válido.
GET /auditoria	Requiere sesión y AUDITORIA_CONSULTAR; acepta filtros por query string.
GET /health	Responde estado HTTP de la aplicación; no demuestra por sí solo que SQL siga disponible.

Para añadir una operación de negocio, mantener el contrato por capas: ruta con acceso requerido, controlador con validación de entrada, servicio con parámetros tipados y un procedimiento SQL autorizado de manera explícita a securefinance_app.



Limitaciones

- express-session usa MemoryStore, previsto solo para uso académico local; no persiste y no escala como almacén de sesiones de producción.
- El esquema contiene Token_Recuperacion y DebeCambiarPassword, pero el sistema no implementa flujo completo de recuperación ni cambio de contraseña en la interfaz.
- No hay configuración de despliegue HTTPS o proxy de producción. La cookie segura depende de la configuración NODE_ENV y la infraestructura de despliegue.
- La semilla admin_demo y los clientes/productos DEMO son datos ficticios de aprendizaje, no cuentas de producción.
- La aplicación enlaza por defecto a 127.0.0.1 y el README presenta una ejecución local.
- USUARIOS_ADMINISTRAR existe como permiso semilla, pero no hay pantalla ni rutas administrativas de usuario.
- La revisión ejecutó pruebas Node, pero no tuvo una instancia SQL Server para verificar scripts, concurrencia, permisos efectivos ni evidencia de base de datos.

Conclusiones

La seguridad separa autenticación, permisos de aplicación y privilegios SQL. SHA2_512 con salt individual reduce la exposición de contraseñas en reposo; RBAC restringe ventas y auditoría en rutas; la cuenta SQL dedicada tiene concesiones por procedimiento y tipo TVP.

La integridad de venta depende de validaciones en SQL, relaciones y restricciones del esquema. BEGIN/COMMIT/ROLLBACK agrupa existencias, factura, detalle y caja; UPDLOCK/HOLDLOCK en orden ascendente protege las filas de productos frente a ventas concurrentes. La ejecución local no confirmó todavía estas propiedades con las suites SQL.

Bitacora_Acceso, Bitacora_Transacciones y los triggers dan trazabilidad de login y de eventos seleccionados. El trabajo integra capas web y base de datos en contratos concretos y deja pruebas automatizadas Node que cubren autorización, validación, CSRF, escape de datos y respuesta HTTP.

El proyecto demuestra el uso conjunto de modelado relacional, claves, procedimientos, tipos de tabla, transacciones, permisos, sesiones y pruebas automatizadas. Para sostener sus afirmaciones sobre persistencia, permisos SQL y rollback, el paso académico pendiente es ejecutar las cuatro suites contra un SQL Server de pruebas y adjuntar capturas originales.

Recomendaciones

- Ejecutar las suites SQL en una base de pruebas nueva y guardar salidas/capturas sin datos secretos.
- Validar que securefinance_app no tenga roles o permisos heredados antes de conectar la aplicación.
- Reemplazar MemoryStore por almacenamiento compartido y configurar HTTPS/proxy de confianza antes de un despliegue multiusuario.
- Diseñar el flujo de cambio y recuperación de contraseña antes de usar el indicador DebeCambiarPassword fuera del entorno académico.
- Mantener las solicitudes de venta sin reintentos automáticos cuando el resultado es incierto; confirmar la factura en el histórico antes de volver a procesarla.
- Actualizar la documentación junto con cambios a los scripts Full/modulares, roles, permisos o pruebas.



Contribuciones del equipo

Integrante	Responsabilidad declarada	Evidencia revisada en repositorio
Kenneth Christian Amaldo Téllez Castillo	Arquitectura, seguridad, usuarios, roles, permisos, autenticación, sesiones, RBAC e integración.	src/config, auth, server.js; database/02-04; commits base, autenticación y paquete final; rama main y merges.
Edward Rene Winter Tiu	Tablas de negocio, clientes, productos, facturación multiproducto, inventario, caja y transacciones ACID.	database/05-06; src/services/ventaService.js, ventaController.js; commit 353cf2a feat: complete transaction and invoicing module; rama feature/edward-transacciones.
Jose Miguel de León Aguilar	Auditoría, funciones, triggers, bitácoras, histórico y reportes.	database/07-08; src/services/auditoriaService.js y vistas; commit 8d7c536 y módulo consolidado 2c0171c; rama feature/jose-auditoria.
Rubén Adolfo Herrera Morales	Documentación técnica, ERD, matriz de pruebas, manual, evidencias y documento final.	README atribuye estas responsabilidades. En main existe referencia PDF previa.